

课程讲义

Language models for autoformalization and theorem proving

CS294/194-280: Advanced Large Language Model Agents

Kaiyu Yang, Meta FAIR

中文教材化讲义 / Codex Harness Build



A blue-themed MOOC slide for Kaiyu Yang. On the left, a circular portrait of Kaiyu Yang is shown. Below it, his name 'KAIYU YANG' and the Meta logo are displayed. On the right, the text 'UC Berkeley Center for Responsible, Decentralized Intelligence' is at the top. Below that, 'Advanced Large Language Model Agents MOOC' is written in white. A horizontal line separates this from 'Language Models for Autoformalization and Theorem Proving'. Another horizontal line follows, with 'April 7th, 2025' below it. A grid of white dots is in the bottom right corner.

UC Berkeley Center for Responsible,
Decentralized Intelligence

Advanced Large Language
Model Agents MOOC

Language Models for
Autoformalization and
Theorem Proving

April 7th, 2025

KAIYU YANG

Meta

课程页: Berkeley RDI SP25 course page

录播: YouTube recording

slides: mathverification.pdf

补充 readings: LeanDojo / Autoformalization with LLMs / Autoformalizing Euclidean Geometry

目录

1 本讲学习目标	2
2 背景与问题设置	2
2.1 为什么“会做数学题”不等于“会 formal reasoning”	2
2.2 两个 gap: advanced math 与 valid proofs	3
3 核心概念与术语	4
4 formal reasoning 的环境视角	4
4.1 proof assistants: 为什么它们是 agent environment	4
4.2 Lean proof state: 从文本生成到交互式搜索	5
4.3 L08 的 AlphaProof 在本讲中的位置	6
5 theorem proving: 从开放基础设施到检索增强系统	6
5.1 LeanDojo: 把 theorem proving 研究变成可复现工程	6
5.2 ReProver: 为什么 theorem proving 需要 retrieval	7
5.3 infinite action space 与 LIPS 的启示	8
6 autoformalization: 把 informal mathematics 送进 formal world	9
6.1 autoformalization 到底在做什么	9
6.2 为什么 autoformalization 难以自动评估	10
6.3 reasoning gaps: proof autoformalization 比 theorem autoformalization 更难	11
6.4 Euclidean geometry: diagrammatic reasoning 为什么特别难	11
6.5 equivalence checking 与 diagrammatic formal system	12
7 关键论文与课程 readings 的连接	13
7.1 LeanDojo: 开放环境的重要性	13
7.2 Autoformalization with Large Language Models: 正式定义任务	13
7.3 Autoformalizing Euclidean Geometry: geometry 是试金石	13
8 例子、反例、失败模式和边界条件	13
8.1 formal reasoning 的典型失败模式	13
8.2 与 verification 的边界	14
9 与前后讲的联系	14
10 本章小结	14
11 复习题	14
12 深入思考题	14
13 延伸阅读	15

1 本讲学习目标

本讲的目标不是简单罗列 theorem proving 论文，而是把形式推理任务拆成一个清晰的概念地图。读完本讲后，读者应当能够回答：

- 为什么 current math LLM 的主流训练 recipe 还不足以解决 formal reasoning。
- formal specification、verification、theorem proving、proof search、autoformalization 各自的输入输出是什么。
- proof assistant 为什么是 advanced agents 的理想 environment 之一。
- LeanDojo、ReProver、LIPS 分别在 theorem proving 栈中解决了什么 bottleneck。
- 为什么 autoformalization 的真正难点在 reasoning gaps、semantic equivalence 和 diagrammatic reasoning。

2 背景与问题设置

2.1 为什么“会做数学题”不等于“会 formal reasoning”

Kaiyu Yang 在开头先回顾了 math/coding arms race，并指出当前 math LLM 的训练 recipe 非常清晰：强预训练模型 + 数学监督数据的 SFT + 在可验证任务上的 RL。其抽象形式可以写成

$$\theta^* = \text{RL}(\text{SFT}(\theta_0; \mathcal{D}_{\text{math}}); \mathcal{R}_{\text{verify}}).$$

这里 θ_0 是 base model， $\mathcal{D}_{\text{math}}$ 是数学数据， $\mathcal{R}_{\text{verify}}$ 是 verification-derived reward。这个 recipe 在 AIME、competition math 或 coding 上很强，但它并不自动给出 formal reasoning 能力。

How LLMs are Trained to Solve Math Problems?

- **Supervised finetuning (SFT):** “Good data is all you need!”
- **Reinforcement learning (RL):** “Verifiability is all you need!”
- Methods are straightforward, but the devil is in the details, e.g., data curation/cleaning, infrastructures for training and inference

Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

7

图 1: 当前 math LLM 的主流训练 recipe。

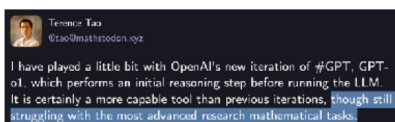
问题在于：很多 benchmark 只要求猜对答案，却不要求写出 machine-checkable 的证明。于是模型也许能在 **answer guessing** 上表现很好，但仍然不会处理 formal theorem statement、proof states、premise retrieval 或 semantic equivalence。

2.2 两个 gap: advanced math 与 valid proofs

Slides 很明确地给出两个 gap。第一，**Pre-college Math** $\rightarrow$ **Advanced Math**: 现有成功大多发生在竞赛或中学以上但仍然高度结构化的问题上。第二，**Guessing Answers** $\rightarrow$ **Writing Proofs**: 从选一个对答案，到构造一条 checker 接受的 formal proof，是质变而不是量变。

Gap 1: Pre-college Math $\rightarrow$ Advanced Math

- Existing successes are mostly on pre-college math, e.g., AIME, IMO
- LLMs struggle with more advanced math, e.g., mathematical research



EPOCH AI

- o3's FrontierMath results come with caveats
- How to solve problems w/o numeric answers?

Clarifying the Creation and Use of the FrontierMath Benchmark

We clarify that OpenAI commissioned Epoch AI to produce 300 math questions for the FrontierMath benchmark. They own these and have access to the statements and solutions, except for a 50-question holdout set.

Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

25

图 2: 从 pre-college math 到 advanced math 的能力 gap。

LLMs Alone are Not Enough

- Current math LLMs rely heavily on data and verifiability
- Data scarcity
 - Limited to data-rich domains, e.g., pre-college math
 - Cannot tackle advanced math or proofs
- Lack of verifiability
 - Solutions can only be evaluated by comparing with the ground truth
 - Limited to problems with numeric solutions, e.g., GSM8K, MATH
 - Not applicable to most problems in advanced math

Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

29

图 3: 仅靠 LLM 文本能力仍然不够。

这也是讲者为什么强调 **formal reasoning is the missing ingredient**。formal reasoning 把数学从“可被文字描述的任务”改写成“可在 formal system 中被检查和搜索的任务”。这一步与 L08 的 AlphaProof 正好对上: 若没有 formal environment, RL 和 search 很难得到可靠反馈; 若没有 theorem-proving machinery, formal environment 又只会停留在静态库。

3 核心概念与术语

- **informal reasoning**: 自然语言层面的数学解释、草图和启发式推理，允许大量默认背景知识。
- **formal specification**: 把问题写成 proof assistant 能接受的 formal theorem statement 或 program specification。
- **verification**: 让 proof checker 或 verifier 判断一条 formal proof / program trace 是否满足规范。
- **theorem proving**: 在形式系统内部构造证明，输出的是 formal proof steps 或 tactics。
- **proof search**: 在 proof states 与 actions 的图上进行系统探索，可能结合 retrieval、pruning、value estimates 或 symbolic rules。
- **autoformalization**: 从 informal theorem 或 informal proof 自动翻译出 formal theorem 或 formal proof。

不要把这些术语压扁成“数学推理”

verification 负责检查，formal specification 负责把对象写对，autoformalization 负责自动生成 formal 表示，theorem proving 负责在 formal system 中找证明，proof search 则是 theorem proving 的算法内核。它们相关，但不能互相替代。

4 formal reasoning 的环境视角

4.1 proof assistants: 为什么它们是 agent environment

讲者把 proof assistant 定义为“用于书写 formal math 和 formal software proofs 的编程语言”。这一定义非常重要，因为它说明 formal reasoning 不是纯文本任务，而是在程序化环境中操作形式对象。

Proof Assistants (Interactive Theorem Provers)

- Programming languages for writing formal math and software

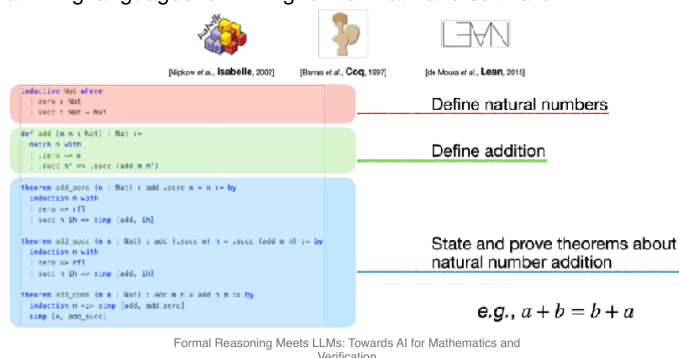
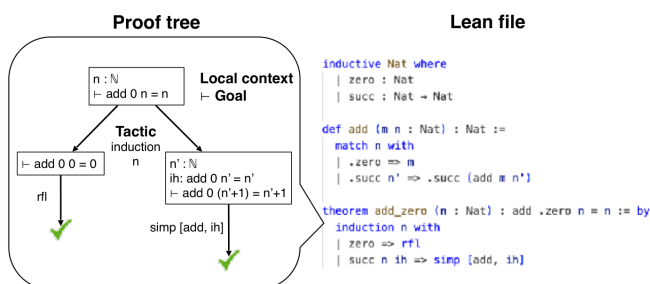


图 4: Proof assistant 是 formal reasoning 的执行环境。

在 Lean 中，定理、假设、proof state 和 tactics 都是显式对象。你不是在写一段让人“看起来像证明”的文字，而是在执行一系列状态转移，让 checker 最终接受目标。

4.2 Lean proof state: 从文本生成到交互式搜索

Formalizing Mathematics in Lean



Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

34

图 5: Lean proof state 将 theorem proving 具体化为 state transition。

这正是 theorem proving 与普通 math QA 的本质差别。一个典型交互循环是：

```
1 while goals remain:
2   inspect the current Lean proof state
3   generate one or more tactics
4   execute a tactic in Lean
5   keep the next state if Lean accepts it; otherwise backtrack
```

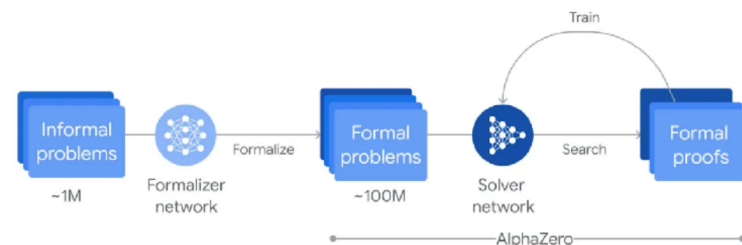
这里已经能看出几层角色划分：

- **verification**: Lean 判断 tactic 合不合法。
- **theorem proving**: 找到能让 goals 全部消失的 tactic sequence。
- **proof search**: 在多条 tactic sequences 之间做探索、回溯和评分。

4.3 L08 的 AlphaProof 在本讲中的位置

Example of AI + Lean: AlphaProof

- Large-scale search and reinforcement learning using feedback from Lean



[Google DeepMind "AI achieves silver-medal standard solving International Mathematical Olympiad problems" 2024]

Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

37

图 6: AlphaProof 作为 AI + Lean 的实例。

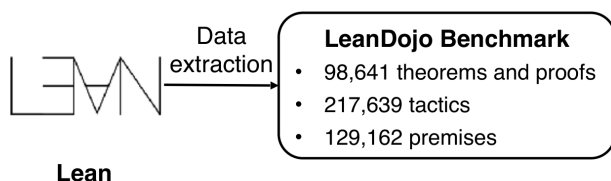
Kaiyu Yang 在这里把 AlphaProof 放进更大的 taxonomy 中：它说明 theorem proving 可以和 RL 深度耦合，但也同时暴露出 formalization 与 domain coverage 的问题。因此 L08 可以被看作“formal reasoning 环境可行”的系统论证，而 L09 更像“formal reasoning 任务族如何拆解”的方法论讲义。

5 theorem proving: 从开放基础设施到检索增强系统

5.1 LeanDojo: 把 theorem proving 研究变成可复现工程

LeanDojo 的价值首先不在模型，而在基础设施。它把 Lean 中的 theorem/proof 数据、premise annotations、benchmark split 和 programmatic interaction 一次性开放出来，从而把 theorem proving 从“只有少数大实验室能做的私有工程”变成可复现研究问题。

LeanDojo



Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

47

图 7: LeanDojo benchmark 的数据规模与开放接口。

这与 harness engineering 的精神完全一致：环境、数据、检查器和交互接口必须先被组织好，agent 才有机会稳定迭代。LeanDojo 做的正是 theorem proving 版本的 “repo as record system”。

5.2 ReProver：为什么 theorem proving 需要 retrieval

Retrieval-Augmented Prover (ReProver)

- Given a state, we retrieve premises from the set of **all accessible premises**

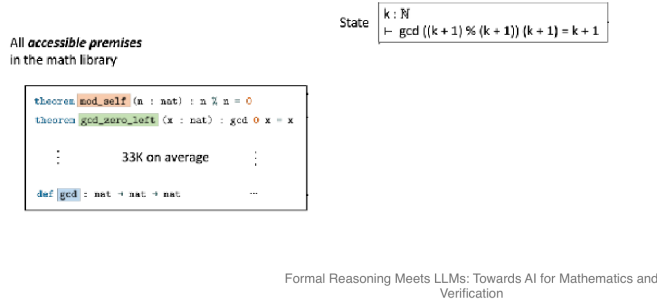


图 8: ReProver 的 retrieval-augmented proving。

给定当前 proof state s ，ReProver 不会盲目生成 tactic，而是先在可访问 premises 集合中做检索，再结合检索结果生成下一步。这可抽象为

$$(a^*, P^*) = \arg \max_{a, P} s_\theta(a \mid s, P) + \lambda r_\eta(P \mid s).$$

其中 P 是检索到的 premises， s_θ 是 tactic generation score， r_η 是 retrieval score。直觉很简单：定理证明不是只在 “下一 token” 上做决策，而是在 “应该引用什么知识” 和 “下一步做什么” 上联合决策。

```

1 given proof state s:
2   retrieve accessible premises P from LeanDojo
3   score premises with a retriever
4   condition the language model on s and top premises
5   generate next tactics and verify them in Lean

```


Summary: A Typical Neural Theorem Prover

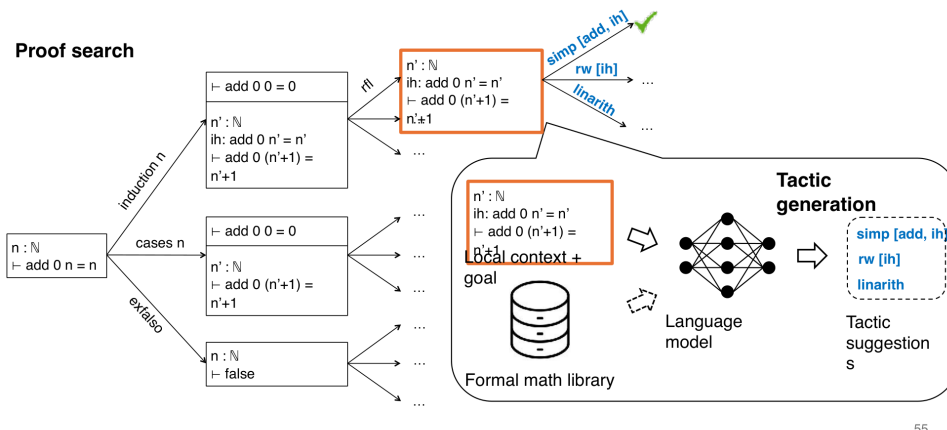


图 9: 典型 neural theorem prover 是多模块系统。

这也是为什么“LLM 直接出 proof”通常不够。真正的系统至少还需要 premise access control、retrieval、search policy 和 checker feedback。

5.3 infinite action space 与 LIPS 的启示

Infinite Proof Search Space

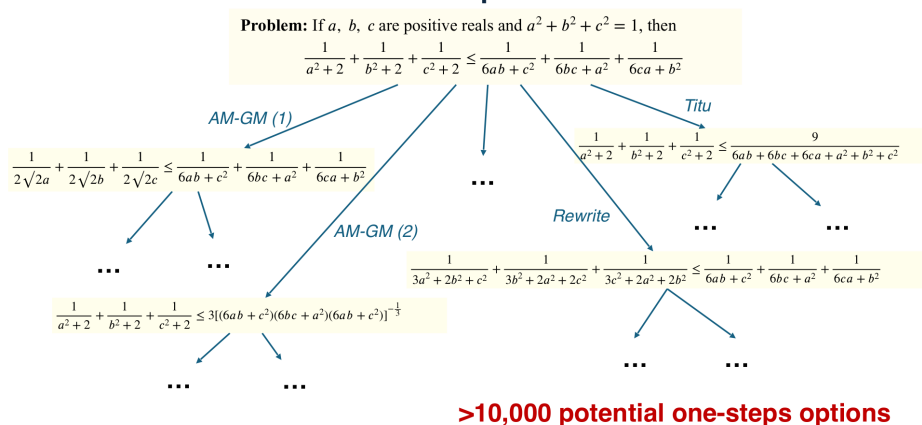


图 10: Theorem proving 的 action space 极大。

Slides 在 inequality proving 段落专门强调：proof search 的动作空间几乎是无限的。很多任务中，模型随时可以发明新的中间变换，而不是只从一个有限 action set 中选一项。这比棋类更接近开放规划问题。

Experimental Results

- Our system LIPS surpasses IMO Gold Medalists in inequality proving

	DeepSeek-R1	Gold medalists	LIPS
#Solved Olympiad-level Inequalities*	4/20	15/20	16/20

* Problems are collected from IMO competitions, national team selection test, training quizzes.

- LIPS achieves SoTA performance across various competition-level datasets

Dataset	# of Problems	Neural Provers			Symbolic Provers		LIPS	Δ
		DSP	MCTS	AIPS [†]	CAD [‡]	MMA [‡]		
ChenNFQ	41	0.0	17.0	-	70.7	68.2	95.1	24.4 [†]
MO-INT	20	0.0	15.0	50.0	60.0	60.0	80.0	20.0 [†]
567NEQ	100	0.0	4.0	-	54.0	52.0	68.0	14.0 [†]
Total	161	0.0	8.6	-	59.0	57.1	76.3	17.3 [†]

[†] The code of AIPS has not been publicly available, we only include its originally reported results.

[‡] CAD and MMA only output verification results, they cannot produce human-readable proofs.

图 11: LIPS 的实验结果。

LIPS 给出的启示是：如果某个子域有强结构，就应该把结构显式写进搜索。它通过把 inequality proving 的步骤分类、做 pruning，并结合符号推理来缩减搜索空间。换言之，通用大模型能力很重要，但 theorem proving 往往还需要 domain structure。

```

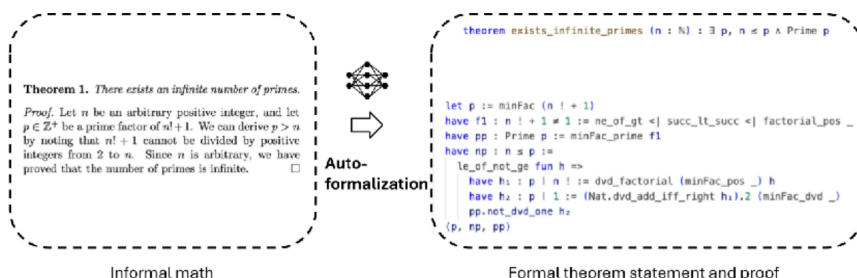
1 generate algebraic transformations with an LLM
2 classify candidates as scaling or pruning steps
3 use symbolic checks to reject inconsistent branches early
4 continue search on the reduced proof space

```

6 autoformalization:把 informal mathematics 送进 formal world

6.1 autoformalization 到底在做什么

Autoformalization



[Wu et al. "Autoformalization with Large Language Models" NeurIPS 2022]

Formal Reasoning Meets LLMs: Towards AI for Mathematics and Verification

68

图 12: Autoformalization 的定义。

讲者把 autoformalization 拆成两类：theorem-level 和 proof-level。可抽象为

$$\hat{T}_{\text{formal}} = g_{\phi}(x_{\text{informal}}), \quad \hat{\pi}_{\text{formal}} = h_{\psi}(x_{\text{informal}}, T_{\text{formal}}).$$

第一项把自然语言 theorem 写成 formal statement；第二项把 informal proof 写成 formal proof。二者都不是 verification，也不是 theorem proving。它们是在构造 theorem prover 所需要的输入对象。

为什么这一步极其关键

没有正确的 formal specification，再强的 theorem prover 也只是在错误问题上努力。autoformalization 决定了系统是否真正理解了原始题意，而不只是会在 formal system 里玩一个别的问题。

6.2 为什么 autoformalization 难以自动评估

Hard to Evaluate Autoformalized Theorems No reliable automatic evaluation

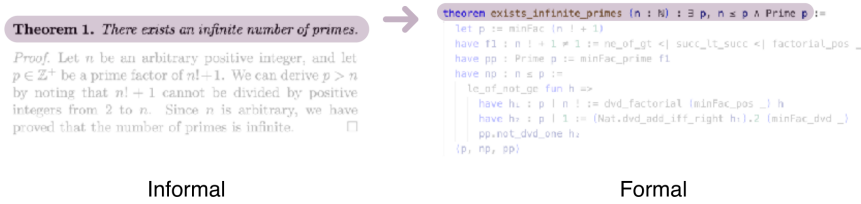


图 13: Theorem-level evaluation 的难点。

autoformalized theorem 的评估难点，在于“表面形式不同但语义相同”的情况太多。一般情形下，很难自动判断一个 formal statement 是否真正等价于原始问题。等价性检查的理想形式是

$$T_1 \equiv T_2 \iff (T_1 \Rightarrow T_2) \wedge (T_2 \Rightarrow T_1).$$

但这要求我们已经能在目标形式系统里证明双向蕴含，本身就不容易。因此 BLEU 之类表面指标没用，纯人工评估又昂贵。

6.3 reasoning gaps: proof autoformalization 比 theorem autoformalization 更难

Reasoning Gaps in Informal Proofs

- Informal proofs have reasoning gaps
 - Explicit gaps: “left to the reader”
 - Implicit gaps
- Formal proofs must be gap-free

Theorem 1. *There exists an infinite number of primes.*

Proof. Let n be an arbitrary positive integer, and let $p \in \mathbb{Z}^+$ be a prime factor of $n!+1$. We can derive $p > n$ by noting that $n!+1$ cannot be divided by positive integers from 2 to n . Since n is arbitrary, we have proved that the number of primes is infinite. $\square$



```
theorem exists_infinite_primes (n : ℕ) : ∃ p, n < p ∧ Prime p :=
  let p := minFac (n! + 1)
  have f1 : n! + 1 ≠ 1 := ne_of_gt <| succ_lt_succ <| factorial_pos _
  have pp : Prime p := minFac_prime f1
  have np : n < p :=
    le_of_not_ge fun h =>
      have h1 : p | n! := dvd_factorial (minFac_pos _) h
      have h2 : p | 1 := (Nat.dvd_add_iff_right h1).2 (minFac_dvd _)
      pp.not_dvd_one h2
  (p, np, pp)
```

Informal

Formal

77

图 14: Informal proofs 普遍存在 reasoning gaps。

自然语言证明经常说“剩余步骤类似”“易证”“由图可见”。人类读者会自动补齐，但 formal proof 不能容忍这些跳步。因此 proof autoformalization 的难点不是把句子翻译成另一种语法，而是要恢复作者默认省略的推理。这一步有时甚至需要重新发明中间 lemma，而不只是抄写。

```
1 informal theorem or proof text
2 -> language model proposes a formal theorem statement
3 -> optional equivalence / semantic checking
4 -> theorem prover fills missing proof steps or rejects the candidate
```

6.4 Euclidean geometry: diagrammatic reasoning 为什么特别难

Autoformalizing Euclidean Geometry

- LeanEuclid: Benchmark for autoformalizing Euclidean geometry
 - 48 from Euclid’s Elements; 125 from UniGeo [Chen *et al.*, **UniGeo**, EMNLP 2022]

81

图 15: LeanEuclid: geometry autoformalization benchmark。

Logical Gaps in Euclid's Proofs

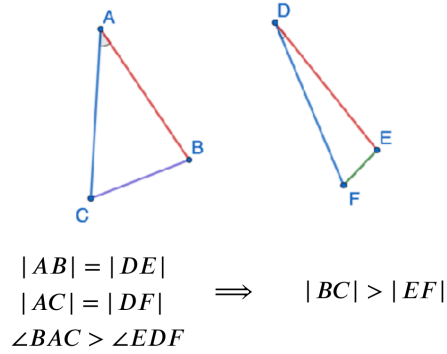
Elements, Book I, Proposition 24

Proposition 24
If two triangles have two sides equal to two sides, respectively, but (one) has the angle encompassed by the equal straight-lines greater than the (corresponding) angle in the other, then the former triangle will also have a base greater than the base of the latter.

Let ABC and DEF be two triangles having the two sides AB and AC equal to the two sides DE and DF , respectively. (That is, AB equal to DE , and AC to DF). Let them also have the angle at A greater than the angle at D . I say that the base BC is also greater than the base EF .

For since angle BAC is greater than angle EDF , let angle EDG , equal to angle BAC , have been constructed at the point D on the straight-line DE [Prop. 1.23]. And let EG be made equal to either of AC or DF [Prop. 1.3], and let EG and FC have been joined.

Therefore, since AD is equal to DE and AC to DG , the two (straight-lines) BA , AC are equal to the two (straight-lines) ED , DG , respectively, also the angle BAC is equal to the angle EDG . Then the base BC is equal to the base EG [Prop. 1.4]. Again since DF is equal to DG , angle EDG is also equal to angle EDF [Prop. 1.5]. Thus EDG (is) greater than EDF . Thus, EFG is much greater than EDF . And since triangle EFG has angle EFG greater than EDF , and the greater angle is subtended by the greater side [Prop. 1.19], side EG (is) thus also greater than EF . But EG (is) equal to BC . Thus, BC (is) also greater than EF .



85

图 16: Euclid proofs 中的 logical gaps。

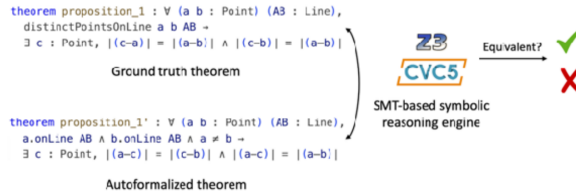
几何证明之所以难，不只因为定理本身复杂，而是因为很多推理建立在图形直觉上。例如“这两条线显然相交在某处”“该三角形显然是等腰”，在人类眼里依赖图即可成立，但 formal system 需要把所有这些条件显式写出来。

这也是《Autoformalizing Euclidean Geometry》那篇 reading 的关键洞见：要想让 LLM 负责 autoformalization，必须先把 diagrammatic information 通过 theorem provers、SMT solvers 或专门 formal system 提取出来，否则语言模型面对的是一个语义残缺的输入。

6.5 equivalence checking 与 diagrammatic formal system

Equivalence Checking Between Theorems

- Two theorems T_1 and T_2 are equivalent iff we can prove $T_1 \iff T_2$
- Symbolic reasoning engine based on SMT solvers



98

图 17: Theorem equivalence checking 的理想形式。

Modeling Diagrammatic Reasoning

The Formal System E

[Avigad *et al.*, "A formal system for Euclid's Elements", 2008]

- Diagrammatic reasoning are logical consequences of "diagrammatic rules"

```
centre_unique : ∀ (a b : Point) (c : Circle), (isCentre c a) ∧ (isCentre c b) → a = b
center_inside_circle : ∀ (a : Point) (c : Circle), isCentre c a → insideCircle a c
inside_not_on_circle : ∀ (a : Point) (c : Circle), insideCircle a c → ¬(onCircle a c)
between_symm : ∀ (a b c : Point), between a b c → (between c b a) ∧ (a ≠ b) ∧ (a ≠ c) ∧ (b ≠ c)
between_needs_line_out : ∀ (a b c : Point) (l : Line), (between a b c) ∧ (onLine a l) ∧ (onLine b l) → onLine c l
between_same_line_in : ∀ (a b c : Point) (l : Line), (between a b c) ∧ (onLine a l) ∧ (onLine c l) → onLine b l
```

- We implement solvers

105

图 18: 把 diagrammatic reasoning 写进 formal system。

Lecture 最后给出的方向不是“再堆更大的 LLM”，而是：针对特定 domain，把原本隐式的 reasoning 规则写进 formal system。例如 geometry 中的图示规则，如果能被建模成 formal rule，那么 autoformalization 和 theorem proving 都会得到更可靠的结构支撑。

7 关键论文与课程 readings 的连接

7.1 LeanDojo: 开放环境的重要性

LeanDojo 对本讲的意义，不只是一个具体模型分数，而是它把 theorem proving 的基础设施、数据和 evaluation 标准化了。没有这样的环境，后续所有 retrieval、search 和 agent-harness 设计都会变得脆弱且不可复现。

7.2 Autoformalization with Large Language Models: 正式定义任务

这篇论文把 autoformalization 作为独立任务立住。它告诉我们，系统必须先学会“把问题写对”，然后 theorem prover 才有机会“把证明找对”。这与 L08 中 manual formalization 仍不可或缺的现实形成直接呼应。

7.3 Autoformalizing Euclidean Geometry: geometry 是试金石

这篇论文的重要性在于，它把 geometry 中的 diagrammatic reasoning 变成可研究对象。它说明 autoformalization 的核心难点不是通用语法翻译，而是恢复隐含前提、处理图示信息并构造可验证的 formal semantics。

8 例子、反例、失败模式和边界条件

8.1 formal reasoning 的典型失败模式

- statement mismatch**: formal theorem 写错，后续 proving 再强也没有意义。
- premise overload**: premise 数量过大，retrieval 失败会让 prover 在错误知识子集上搜索。

- **action explosion**: proof search 分支过多，局部采样无法稳定前进。
- **semantic evaluation gap**: 即使 checker 接受某个 theorem，也未必说明它忠实表达了原始问题。
- **diagrammatic omission**: 几何中的图形约束若未显式补齐，autoformalization 会系统性缺信息。

8.2 与 verification 的边界

verification 的作用是判断一个 formal object 是否成立；它不负责生成 theorem statement，不负责选择 premises，也不负责决定 search policy。因此“有 verifier”不等于“formal reasoning 问题已经解决”。这和安全章节里常见的一个教训类似：**checker 能发现部分错误，但不能替代建模与搜索本身。**

9 与前后讲的联系

L08 展示的是 formal mathematics 作为环境的吸引力；L09 则把这个环境内部的任务进一步拆开，说明其中哪些部分是 RL/search 问题，哪些是表示和语义问题。再往后看，这种“环境 + 表示 + 搜索 + 验证”的拆法，也会在安全 agent、coding agent 和 abstraction/discovery 章节中反复出现。

10 本章小结

本讲最大的贡献，是把“数学推理”从一个模糊标签拆成了多层任务：formal specification、verification、theorem proving、proof search、autoformalization。它同时给出三个关键结论：第一，proof assistant 提供了极其重要的环境反馈；第二，retrieval 与 domain structure 对 theorem proving 仍然关键；第三，autoformalization 的最大难点在于 informal proofs 中无处不在的语义缺口与图形推理。

11 复习题

1. 为什么当前 math LLM 的 recipe 不能自动解决 formal reasoning?
2. formal specification 和 theorem proving 的输入输出分别是什么?
3. ReProver 中 retrieval 的作用是什么?
4. theorem equivalence checking 为什么困难?
5. geometry autoformalization 为什么比普通符号翻译更难?

12 深入思考题

1. 如果 verification 完美，但 theorem statement 经常写错，系统会在什么环节失真?
2. theorem proving 是否更像 coding agent、web agent，还是更像 game-playing agent? 为什么?
3. 针对特定数学子域引入专门 formal system，是否会破坏通用性? 应如何权衡?

13 延伸阅读

- *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*
- *Autoformalization with Large Language Models*
- *Autoformalizing Euclidean Geometry*
- Formal Mathematical Reasoning position paper